

Week 2 - Day 10: Time Complexity

Today, we will delve into the concept of **time complexity**, which is crucial in understanding the efficiency of algorithms. Time complexity describes the amount of computer time taken by an algorithm to run, **as a function of the size of the input to the program**. It is usually expressed in Big-O notation, which describes the worst-case scenario. This notation helps us analyze the execution time required or the space used (e.g., in memory or on disk) by an algorithm.

Topics to be covered:

- Introduction to Time Complexity and Big-O Notation
- Analyzing Time Complexities for Different Algorithms
- Applying Time Complexity Concepts to Real-World Problems

Let's take a few common examples to illustrate time complexity:

Constant Time

An algorithm is said to have a constant time when **it is not dependent on the input data size**. Whether your input data size is 1 or 1000 or 1 million, the time taken to compute the result is constant.

```
def get_first_item(items):  
    return items[0]
```

In the above Python function, no matter how large the list `items` is, the function would take the same amount of time to get the first item in the list. This is thus $O(1)$ time complexity.

Linear Time

An algorithm is said to have a linear time complexity when the running time **increases linearly** with the size of the input data.

```
def linear_search(items, target):  
    for i in range(len(items)):  
        if items[i] == target:  
            return i  
    return -1
```

In the function above, in the worst-case scenario (if the target is at the end of the list or not in the list at all), the function will have to look at each item in the list once. Therefore, the time complexity is $O(n)$, where n is the number of items in the list.

Quadratic Time

An algorithm is said to have a quadratic time complexity when the time execution is proportional to the square of the input size. A common example of this is the nested for loop.

```
def bubble_sort(items):
    n = len(items)
    for i in range(n):
        for j in range(0, n - i - 1):
            if items[j] > items[j + 1]:
                items[j], items[j + 1] = items[j + 1], items[j]
```

In the bubble sort function above, for each item in the list, we're potentially looking at every other item in the list. As such, this gives us a time complexity of $O(n^2)$.

Visualizing time complexity

Above is a plot of the different time complexities as a function of n (input size).

Exercise 1: Adding Numbers []

Consider the following function that returns the sum of the sequence of numbers from 0 to n (exclusive).

```
def add_nums(n):
    answer = 0
    for i in range(n):
        answer += i
    return answer
```

1.1 For how many iterations is the loop going to run?

1.2 What is the time complexity of the code in terms of Big-O notation?

Now, consider another function `add_num_list(lst)`, which calls `add_nums` for each element in the list `lst`.

```
def add_num_list(lst):
    for i in lst:
        add_nums(i)
```

1.3 How many times does the `add_num_list` function call `add_nums`?

1.4 Now, how many addition operations are going to be performed in total?

1.5 What is the time complexity of the code in terms of Big-O notation?

Exercise 2: Image Filters [] []

Recall the code for filters on images. Here is the filter that paints a red gradient:

```
canvas = SimpleImage.blank(600, 255)
for column in range(canvas.width):
    for row in range(canvas.height):
        # If we want to make sure this works for any size image:
        red = int((column / canvas.width) * 255)
        green = 0
        blue = 0
        canvas.set_rgb(column, row, red, green, blue)
canvas.show()
```

2.1 If the width and height of the image is given by n , what is its time complexity?

1. $O(1)$
2. $O(\log n)$
3. $O(n)$
4. $O(n^2)$
5. $O(n^3)$

Now, consider the following code for drawing a horizontal line:

```
for col in range(10):
    canvas.set_rgb(col, 0, 255, 0, 0)
canvas.show(resize_width=200)
```

2.2 What about this program that draws a horizontal line across the canvas?

1. $O(1)$
2. $O(\log n)$
3. $O(n)$
4. $O(n^2)$
5. $O(n^3)$

Exercise 3: Analyzing Code [] [] []

Give the worst-case Big-O time complexity of each of the following functions. All of the below functions take a list L as input. Your answers can be either $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^2)$, and $O(n^3)$. You should assume that functions `constantfun` and `linearfun`, when run on lists of n elements, have worst-case time complexity $O(1)$ and $O(n)$, respectively.

You may assume the `len` function in Python runs in constant time. Make sure to justify each of your answers.

3.1 What is the worst-case time complexity of `fun1`?

```
def fun1(L):
    n = len(L)
    constantfun(L)
    for i in range(0, n // 10):
        linearfun(L)
```

3.2 What is the worst-case time complexity of `fun2`?

```
def fun2(L):
    n = len(L)
    for i in range(0, 20000):
        for j in range(0, 20000):
            constantfun(L)
            constantfun(L)
```

3.3 What is the worst-case time complexity of `fun3`?

```
def fun3(L):
    n = len(L)
    for i in range(0, n - 3):
        constantfun(L)
    for j in range(0, n - 3):
        constantfun(L)
    constantfun(L)
```

3.4 What is the worst-case time complexity of `fun4`?

```
def fun4(L):
    n = len(L)
    while n > 0:
        constantfun(L)
        n = n // 3
```

Exercise 4: Analyzing More Code [] [] []

4.1 What is the time complexity of `fun`?

```
def fun(n):
    if n <= 1:
        return 1
```

```
else:
    return fun(n-1) + fun(n-1)
```

4.2 What is the time complexity of fun?

```
def fun(n):
    i = 1
    while i < n:
        i = i * 2
```

4.3 What is the time complexity of fun?

```
def fun(arr):
    n = len(arr)
    result = [0] * n
    for i in range(n):
        result[i] = max(arr[:i+1])
    return result
```

Exercise 5: Linear Search []

Implement a function called `linear_search` that takes in two parameters: a list of integers `lst` and a target integer `target`. The function should return the index of the target integer if it is found in the list, or `-1` if it is not present.

Examples:

```
linear_search([4, 2, 7, 1, 9, 5], 7) == 2
linear_search([4, 2, 7, 1, 9, 5], 3) == -1
linear_search([], 0) == -1

# TODO: Implement linear search
def linear_search(lst, target):
    # your code here
    pass

# Use this cell to test your solution
assert linear_search([1, 2, 3, 4, 5], 3) == 2
assert linear_search([1, 2, 3, 4, 5], 6) == -1
assert linear_search([1, 2, 3, 4, 5], 1) == 0
assert linear_search([1, 2, 3, 4, 5], 5) == 4
assert linear_search([1, 2, 3, 4, 5], 4) == 3
assert linear_search([4, 2, 7, 1, 9, 5], 7) == 2
assert linear_search([4, 2, 7, 1, 9, 5], 3) == -1

print("Test cases passed. []")
```

Congratulations 🎉